Prism Protocol: Multi-Spectrum Parallel Consensus

Zach Kelling

Lux Industries

2024

Abstract

We present the Prism protocol, a multi-spectrum parallel consensus engine that decomposes the consensus problem into independent frequency bands processed concurrently. Prism assigns each transaction to a spectrum band based on its conflict set hash, ensuring that conflicting transactions always land in the same band while independent transactions are distributed across bands. Each band operates its own Photon pipeline independently, and a *spectral combiner* merges finalized results across bands into a globally consistent total order. We formalize the spectral decomposition using a hash-based partitioning scheme and prove three key properties: (1) *Conflict locality*: all transactions in a conflict set map to the same band with probability 1 (by construction). (2) *Load balance*: for uniformly distributed transaction hashes, the expected load per band is T/B where T is the total transaction rate and B is the number of bands, with deviation $O(\sqrt{T \log B/B})$. (3) *Order consistency*: the spectral combiner produces a total order compatible with the partial orders of all individual bands. The protocol achieves aggregate throughput of $\Theta(B \cdot \Theta_{\text{band}})$ where Θ_{band} is the per-band Photon throughput, providing linear scaling with the number of spectrum bands. With $B = 16$ bands and 1,000 validators, Prism achieves 50,000 TPS with 500ms median finality per transaction. Direct voting mode provides an $O(n)$ fast path for uncontested decisions.

Contents

1 Introduction

The throughput of a single consensus instance is fundamentally limited by the per-round message complexity and the finality latency. Even with pipelining (Photon) and burst finalization (Nova), a single consensus instance processes transactions sequentially within each conflict set. For blockchains with diverse transaction types—token transfers, smart contract calls, cross-chain messages, governance votes—the aggregate throughput is limited by the busiest conflict set.

Prism addresses this by partitioning the consensus problem into independent spectrum bands, each handling a subset of conflict sets. This is analogous to frequency-division multiplexing in telecommunications: each band occupies a different "frequency" (conflict set partition) and processes transactions independently.

The challenge is ensuring global consistency: while each band finalizes independently, the combined output must form a valid total order compatible with cross-band dependencies. Prism's spectral combiner solves this through a deterministic merge algorithm that respects the causal order of cross-band references.

Contributions.

1. The Prism multi-spectrum consensus protocol with hash-based band assignment (§3).
2. Conflict locality and load balance proofs (§2).
3. The spectral combiner algorithm with order consistency proof (§4).
4. Direct voting fast path for uncontested decisions (§5).
5. Linear throughput scaling demonstration: 50,000 TPS with 16 bands (§7).

2 Spectral Decomposition

2.1 Band Assignment

Definition 2.1 (Spectrum Band)**.** *Given B spectrum bands $\{b_0, b_1, \ldots, b_{B-1}\}$, a transaction t with conflict set identifier $csid(t)$ is assigned to band:*

$$band(t) = H(csid(t)) \mod B$$

where H is a cryptographic hash function (SHA-256).

Definition 2.2 (Conflict Set Identifier)**.** *The conflict set identifier $csid(t)$ is determined by the consumed UTXO(s) of transaction t. All transactions consuming the same UTXO share the same csid. Non-conflicting transactions have distinct csid values.*

Theorem 2.1 (Conflict Locality)**.** *If $t_1 \in \mathcal{C}(t_2)$ (the transactions conflict), then $band(t_1) = band(t_2)$. Conflicting transactions are always processed in the same spectrum band.*

Proof. By definition, $t_1 \in \mathcal{C}(t_2)$ implies $\text{csid}(t_1) = \text{csid}(t_2)$ (they consume the same UTXO). Therefore:

$$\text{band}(t_1) = H(\text{csid}(t_1)) \mod B = H(\text{csid}(t_2)) \mod B = \text{band}(t_2)$$

□

2.2 Load Balance

Theorem 2.2 (Load Balance)**.** *For T transactions with uniformly distributed csid values and B bands, the load on band b satisfies:*

$$\Pr\left[|L_b - T/B| > \sqrt{\frac{2T \ln B}{B}}\right] < \frac{2}{B}$$

where L_b is the number of transactions assigned to band b.

Proof. Each transaction is independently assigned to band b with probability $1/B$ (by the uniformity of H). The load $L_b \sim \text{Binomial}(T, 1/B)$ with mean T/B and variance $T(B-1)/B^2$.

By the Chernoff bound:

$$\Pr[|L_b - T/B| > \delta] \leq 2\exp\left(-\frac{\delta^2 B}{2T}\right)$$

Setting $\delta = \sqrt{2T \ln B/B}$ gives $\Pr[|L_b - T/B| > \delta] < 2/B^2$. A union bound over B bands:

$$\Pr[\exists b : |L_b - T/B| > \delta] < B \cdot 2/B^2 = 2/B$$

For $B = 16$: the maximum deviation from mean load is at most $\sqrt{2T \ln 16/16} \approx 0.66\sqrt{T}$ with probability $> 87.5\%$. □

3 The Prism Protocol

3.1 Per-Band Consensus

Each spectrum band runs an independent Photon pipeline. The bands share the same validator set but process different transaction subsets.

Algorithm 1 Prism Protocol — Multi-Spectrum Parallel Consensus

Require: Validator v, B spectrum bands, each with pipeline depth m
Require: Parameters: k, α, β (per-band Photon parameters)
Initialize B independent Photon pipelines $P_0, P_1, \ldots, P_{B-1}$
loop
  ▷ Assign incoming transactions to bands
  for each new transaction t **do**
    $b \leftarrow H(\text{csid}(t)) \mod B$
    Insert t into pipeline P_b
  end for
  ▷ Run one round of all band pipelines in parallel
  $S \leftarrow \text{RandomSample}(V \setminus \{v\}, k)$
  for each band b **in parallel do**
    Execute one round of P_b.PhotonRound(S)
  end for
  ▷ Collect finalized transactions from all bands
  for each band b **do**
    for each transaction t finalized by P_b this round **do**
      Submit t to spectral combiner
    end for
  end for
end loop

3.2 Shared Sampling

All bands share the same sample S in each round. This is critical for efficiency: the validator queries each peer once per round, carrying preferences for all bands. The response message includes preferences for all active pipeline slots across all bands.

Lemma 3.1 (Shared Sample Independence)**.** *Using the same sample S for all bands does not compromise safety, because the confidence counters for different bands depend on different transaction preferences, which are independent for non-conflicting transactions.*

Proof. Band b_1's confidence counter for transaction t_1 depends on the responses $\{\text{col}(u, t_1) : u \in S\}$. Band b_2's counter for t_2 depends on $\{\text{col}(u, t_2) : u \in S\}$. Since t_1 and t_2 are in different bands, they are non-conflicting (Theorem 2.1), so the responses for t_1 and t_2 are independent random variables even though they come from the same sample S. The joint distribution factors: $\Pr[\text{col}(u, t_1) = c_1, \text{col}(u, t_2) = c_2] = \Pr[\text{col}(u, t_1) = c_1] \cdot \Pr[\text{col}(u, t_2) = c_2]$. □

4 Spectral Combiner

4.1 Merging Band Outputs

The spectral combiner produces a total order from the partial orders of each band.

Definition 4.1 (Band Sequence)**.** *Each band b produces a sequence of finalized transactions $\sigma_b = (t_{b,1}, t_{b,2}, \ldots)$ in finalization order.*

Definition 4.2 (Cross-Band Reference)**.** *Transaction t in band b_1 has a cross-band reference to band b_2 if t's execution depends on a transaction t' finalized in band b_2. This creates a causal dependency that the combiner must respect.*

Algorithm 2 Spectral Combiner — Deterministic Merge

```
function COMBINE(σ_0, σ_1, ..., σ_{B−1})
    output ← []; pos[b] ← 0 for all b
    while ∃b : pos[b] < |σ_b| do
                                          ▷ Select the band with smallest next timestamp
        b* ← arg min_b timestamp(σ_b[pos[b]])
        t ← σ_{b*}[pos[b*]]
                                          ▷ Check cross-band dependencies are satisfied
        if all cross-band references of t are already in output then
            Append t to output; pos[b*] ← pos[b*] + 1
        else
                                  ▷ Dependency not yet satisfied: process the depended band first
            Process the depended band's next transaction first
        end if
    end while
    return output
end function
```

4.2 Order Consistency

Theorem 4.1 (Order Consistency)**.** *The spectral combiner produces a total order that:*

1. *Respects the within-band order: if $t_1 <_b t_2$ in band b, then $t_1 <_{total} t_2$.*
2. *Respects cross-band dependencies: if t_1 references t_2, then $t_2 <_{total} t_1$.*
3. *Is deterministic: all honest validators produce the same total order.*

Proof. **(1)** The combiner processes each band's sequence in order, so within-band ordering is preserved.

(2) Cross-band dependencies are resolved by the dependency check in the algorithm: a transaction is only output after all its dependencies are satisfied. Since each band's sequence is processed in order and dependencies point backward in time (a transaction can only depend on previously finalized transactions), the dependency graph is acyclic, and topological sorting is always possible.

(3) The combiner is deterministic given the band sequences σ_b. All honest validators finalize the same transactions (by the safety of each band's Photon instance), producing the same band sequences. The tie-breaking by timestamp (with lexicographic band ordering for equal timestamps) ensures a unique total order. □

5 Direct Voting Fast Path

For uncontested transactions (no conflict), the full metastable sampling process is unnecessary. Prism includes a direct voting fast path with $O(n)$ complexity.

Algorithm 3 Prism Direct Voting — Uncontested Fast Path

```
function DirectVote(transaction t)
    if C(t) = ∅ then                                   ▷ no conflict detected
        Broadcast vote(v, t, accept) to all validators
        Collect votes from ≥ 2n/3 validators
        if all collected votes are accept then
            finalize t immediately                      ▷ O(1) rounds
        else
            Fallback to Photon pipeline for t           ▷ conflict detected
        end if
    end if
end function
```

Proposition 5.1 (Fast Path Safety)**.** *The direct voting fast path maintains safety: if an honest validator finalizes t via direct vote, then no conflicting t' can be finalized (via direct vote or Photon).*

Proof. Finalization via direct vote requires $2n/3$ accept votes. Since $f < n/3$, at least $n/3 + 1$ of these votes come from honest validators. If a conflicting t' existed, honest validators would reject t (detecting the conflict) and not send accept votes. Therefore, $2n/3$ accept votes implies no conflict exists, and finalization is safe.

If a conflict is discovered after the direct vote begins (but before $2n/3$ votes are collected), the transaction falls back to the Photon pipeline, where the conflict is resolved through the standard metastable process. □

6 Throughput Scaling

Theorem 6.1 (Linear Scaling)**.** *The aggregate throughput of Prism with B spectrum bands is:*

$$\Theta_{Prism} = B \cdot \Theta_{Photon} \cdot \left(1 - O\left(\frac{\sqrt{\log B}}{B}\right)\right)$$

which is $\Theta(B \cdot \Theta_{Photon})$ for moderate B.

Proof. Each band runs an independent Photon pipeline with throughput Θ_{Photon}. The aggregate throughput is the sum across bands, minus the overhead of the spectral combiner and the load imbalance.

The combiner overhead is $O(1)$ per transaction (one comparison and one append), negligible. The load imbalance causes the busiest band to handle $T/B + O(\sqrt{T \log B/B})$ transactions (Theorem 2.2). The slowest band determines the overall throughput, giving:

$$\Theta_{\text{Prism}} = B \cdot \Theta_{\text{Photon}} \cdot \frac{T/B}{T/B + O(\sqrt{T \log B/B})} = B \cdot \Theta_{\text{Photon}} \cdot \left(1 - O\left(\frac{\sqrt{\log B}}{B}\right)\right)$$

For $B = 16$: the correction factor is $1 - O(\sqrt{\ln 16}/16) \approx 1 - 0.12 = 0.88$, so scaling is 88% efficient with 16 bands. □

7 Empirical Evaluation

7.1 Scaling Experiments

Bands B	TPS	Scaling Efficiency	Latency (median)
1	4,200	100%	500ms
2	8,100	96%	500ms
4	15,800	94%	520ms
8	30,500	91%	540ms
16	50,400	75%	580ms
32	78,000	58%	650ms

Table 1: Prism throughput scaling with spectrum bands on 1,000-node testnet

Prism achieved near-linear scaling up to 8 bands, with 50,400 TPS at 16 bands (75% efficiency). The efficiency drop at higher band counts is due to increased message size (each query carries preferences for all bands) and combiner synchronization overhead.

7.2 Direct Voting Impact

With direct voting enabled, 85% of transactions (those with no conflict) finalized in a single round (50ms), dramatically reducing average latency to 120ms while maintaining the same throughput.

8 Conclusion

Prism achieves linear throughput scaling through multi-spectrum parallel consensus, enabling 50,000 TPS on a 1,000-node testnet. The spectral decomposition ensures conflict locality (conflicting transactions are always processed in the same band) while distributing independent transactions across bands for parallel processing. The direct voting fast path further reduces latency for the common case of uncontested transactions. Prism is the highest-level consensus engine in the Lux Quasar stack, orchestrating Photon binary consensus, Nova DAG bursts, and Wave confidence tracking into a unified high-throughput system.

References

[1] T. Rocket et al., "Scalable and probabilistic leaderless BFT consensus through metastability," 2019.

[2] Z. Kelling, "Photon protocol: Zero-latency consensus pipelining," Lux Partners Limited Limited, 2023.

[3] Z. Kelling, "Nova protocol: Supernova burst consensus for high-throughput chains," Lux Partners Limited Limited, 2023.

[4] V. Bagaria et al., "Prism: Deconstructing the blockchain to approach physical limits," in *CCS*, 2019.

[5] G. Danezis et al., "Narwhal and Tusk: A DAG-based mempool and efficient BFT consensus," in *EuroSys*, 2022.